



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Tècnica Superior d'Enginyeria
de Telecomunicació de Barcelona



Work-flux optimization for the use of GRASP algorithm in quasi-real-time

Master Thesis
submitted to the Faculty of the
Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona
Universitat Politècnica de Catalunya
by
Mireia López Barrón

In partial fulfillment
of the requirements for the master in
(*Write the name of your Master*) **ENGINEERING**

Advisor: name of the advisor
Barcelona, Date XXXXX



Contents

List of Figures	3
List of Tables	3
1 Introduction	6
1.1 Gantt Diagram	6
1.2 Deviation from the initial plan	7
2 Implementation	8
2.1 Set up development environment	8
2.2 Views and ViewModels	9
2.3 Controllers	10
2.4 Polymorphism through interfaces	11
2.4.1 Factory pattern	11
2.4.2 Project Actions	12
2.4.3 Download Controllers	13
2.4.4 Matlab Scripts Implementations	13
2.5 Interactions with other applications	14
2.6 Code organization	15
3 Testing	17
4 Conclusions	19
References	20
Appendices	22
A Earlinet Service Class	22
B Aeronet Service Class	22
C RunMatlabScript method using ProcessStartInfo class	23
D Test Results	24

List of Figures

1	Project's Gantt diagram	7
2	extensions	8
3	Application interactions diagram	15
4	Code organization example	16
5	Implemented Unit Tests	18

Listings

List of Tables

Revision history and approval record

Revision	Date	Purpose
0	10/12/2025	Document creation
1	dd/mm/yyyy	Document revision

DOCUMENT DISTRIBUTION LIST

Name	e-mail
[Student name]	
[Project Supervisor 1]	
[Project Supervisor 2]	

Written by:		Reviewed and approved by:	
Date	dd/mm/yyyy	Date	dd/mm/yyyy
Name	Xxxxxxxx yyyyyyy	Name	Zzzzzzzz Wwwwwww
Position	Project Author	Position	Project Supervisor

Abstract

Every copy of the thesis must have an abstract. An abstract must provide a concise summary of the thesis. In style, the abstract should be a miniature version of the thesis: short introduction, a summary of the results, conclusions or main arguments presented in the thesis. The abstract may not exceed 150 words for a Degree's thesis.

1 Introduction

Recently, the need to analyse and monitor atmospheric aerosols has grown significantly, as their negative impact has grown significantly in different sectors: human health, climatology and air transport. GRASP (Generalized Retrieval of Atmosphere and Surface Properties), which was introduced by Dubovik, is the first unified algorithm used for obtaining atmospheric properties from different remote sensing observations including satellite, ground-based and airborne measurements, both passive and active, of atmospheric radiation and their combinations[25].

The main objective of this work is to develop a software that automates the full GRASP execution process, which includes:

1. Download photometer data from the Aeronet website
2. Download Lidar (Light Detection and Ranging) data from the ACTRIS-EARLINET website
3. Combine the data downloaded from both websites
4. Run GRASP
5. Plot the obtained results

1.1 Gantt Diagram

The time organization of the project is shown in the Gantt diagram of Figure 1 representing a time lapse of one semester. It is important to note that even though there are different groups of tasks for implementation and testing, tests have been executed during the implementation phase to ensure the correct functionality of the different steps.

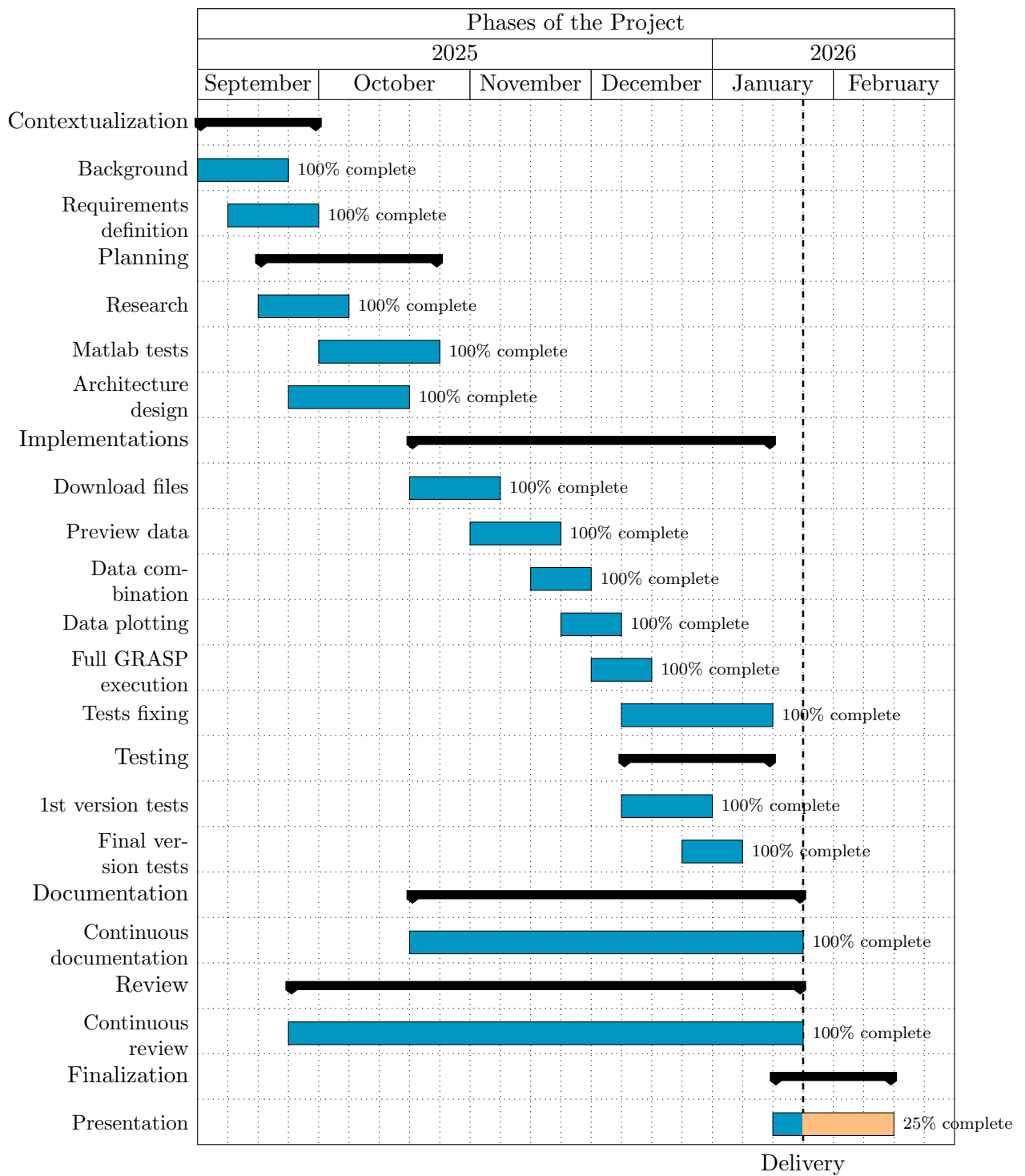


Figure 1: Gantt diagram of the project

1.2 Deviation from the initial plan

2 Implementation

The code implemented for this project can be found in the GitHub repository[22]. Some implementations have been explained previously in other sections of this document like the Matlab Controller or the Aeronet Download Service, but this section aims to explain the purpose of the different classes and the different design patterns that have been used in order to follow the SOLID principles as much as possible.

2.1 Set up development environment

In order to create this project, it is necessary to meet several requirements to have a correct development environment. First of all, it is necessary to choose a correct IDE. There are different options, including JetBrains Rider and Visual Studio Code, but for this project, it has been decided to use Visual Studio Community [17].

The application will be run using .NET, which is a free and open-source application platform supported by Microsoft used for developing different type of applications using C# [19] and which is usually already installed in the system[18].

In order to use Avalonia, it is necessary to install the corresponding extension for Visual Studio. It can be done by accessing to `Extensions\ManageExtensions`:

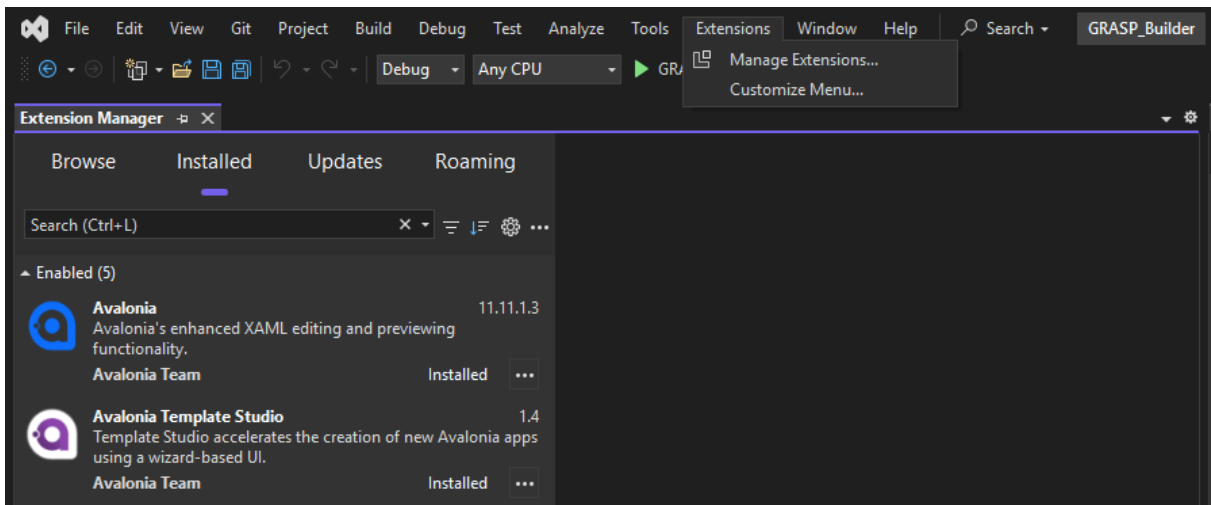


Figure 2: Extensions installed in Visual Studio

Since the app will be executing Matlab scripts, it is also necessary to have Matlab installed. In this case it has been used Matlab R2024a[20].

In order to run the GRASP algorithm it is necessary to access to CALCULA, where it is already installed and running in Ubuntu that will only be used for testing.

2.2 Views and ViewModels

In order to not overload the content of the different files for the User Interface implementation (Views and Windows) and management (View Models) it has been created a different View class for each Tab and for each Window, that can be resumed in the following list:

- About Window: Window that shows general information about the app and containing button "Help" which opens User Manual
- Data Combination View: Tab where the user can select Measure ID to combine data, showing available configurations for selected date and time and executing GRASP.
- Data Download View: Tab where the user can select dates and station and download corresponding data from Earlinet and Aeronet websites.
- Home View: Shows the following action options: to create, open or import a project.
- Home Project Action Window: Window that appears when the user clicks on a project action option in Home View and is in charge of performing it.
- Main Window: Window that contains all the tabs and shows them when necessary. It can show the Home View when no project is loaded or a combination of the other tabs with an additional Log and a Menu in order to manage the app Settings or the current project.
- Message Window: Window that shows a message to the user that can be an Error, a Warning or just information.
- Log View: Tab that shows the Log content of the app.
- Plot View: Tab where the user can select and create the desired figures after executing GRASP algorithm.
- Settings View: Window where the user is able to modify the app configuration parameters.

All these classes for both Views and ViewModels are implemented using the MVVM pattern using ObservableObject and RelayCommand classes from the .NET community toolkit. In addition, all ViewModels classes have been structured by regions in order to homogeneously group the different properties, commands and methods.

In order to establish communication between different classes to notify modifications, it has been used the Messenger class from the .NET community toolkit. It consists on defining a Send Action, that can include some parameter or variable and a Receive Action, that will be executed when the Send Action is called, executing the linked method.

In the following example it can be seen how the DataCombinationViewModel class has been implemented using the Messenger class to receive modifications in the buttons enabled state:

```
1 public DataCombinationViewModel()  
2 {
```

```
3     Messenger.Default.Register<bool>("UpdateButtonsEnabled",
4         UpdateButtonsEnabled);
5 }
6 private void UpdateButtonsEnabled(bool status)
7 {
8     AreButtonsEnabled = status;
9     if (!status)
10    {
11        isEnabled_D1_L = false;
12        isEnabled_D1P_L = false;
13        isEnabled_D1_L_VD = false;
14        isEnabled_D1P_L_VD = false;
15    }
16 }
```

In the corresponding controller, in order to send the modification in order to enable the buttons, for example, it is necessary to execute the following line:

```
1 Messenger.Default.Send<bool>("UpdateButtonsEnabled", true);
```

Using Messenger allows to notify modifications in the App between classes that exist simultaneously, in this case between the different Tabs and between ViewModels and Controllers. In the case of creating new Windows like the Settings one, it is not necessary, since the needed information can be passed as parameters to the Constructor when it is initialized.

2.3 Controllers

- Download Controllers: Different controllers have been developed for each different web site. They both implement the IDownloadController Interface. Each controller is responsible for the different actions that need to be executed when downloading the respective data by managing their respective web services.
- CmdController: This controller is responsible for the different actions needed to execute commands, in this case, necessary to execute the GRASP algorithm via the command line. Since this application have been designed to work with CALCULA, command line for executing GRASP only is executed with Ubuntu syntax.
- AppConfig: This class manages the application configuration, obtaining and saving the different configuration parameters from the configuration file (config.json), located in the execution directory.
- Helpers: Different helper classes have been developed to perform different actions. FormatHelpers are used for changing the format of variables, for example: Converting a dictionary to a string variable. FileHelpers are used for managing the files and directories renaming, copying, etc.
- Logger: This class is responsible for managing the logging actions.

- MessagesController: This controller is responsible for managing the different messages that can be shown to the user: Errors, warnings and information messages.
- ProjectConfig: This class manages the project configuration, obtaining and saving the different configuration parameters from the project file (project.grasp), located in the project directory.
- MatlabScriptController: This controller is responsible for managing the different actions needed to execute any MatlabScript.

2.4 Polymorphism through interfaces

With the objective of applying the Open/Closed Principle (OCP) and the Interface Segregation Principle (ISP), different interfaces have been implemented in order to be used in different points of the project. Using polymorphism, enables the possibility of using objects of a derived class as objects of a base class[23].

Polymorphism can be implemented with a parent or base class, which defines the common properties and implements virtual methods. Derived classes can use the already implemented methods of the parent class, or override them.

Another possible implementation of polymorphism is through the use of abstract classes or interfaces, which is the one used in this project. Interfaces are contracts that define a set of methods that a class must implement. This enables the possibility to work with different classes, where each class implements the interface in a different way but all classes and actions are used uniformly by calling the interface methods.

2.4.1 Factory pattern

Factory pattern is a creational design pattern that provides an interface for creating objects of a parent class, but deciding which subclass to instantiate[24]. This allows to resume all the different conditions in one single method, instead of using multiple if-else conditions during the development of the code.

An example of this pattern can be seen in the following example, showing the implementation of the DownloadControllerFactory Create method:

```
1 public static IDownloadController Create(DownloadType type, string
   _repositoryDirectory, string _workingDirectory)
2 {
3     switch (type)
4     {
5         case DownloadType.Earlinet:
6             return new EarlinetDownloadController(_repositoryDirectory,
               _workingDirectory);
7         case DownloadType.Aeronet:
8             return new AeronetDownloadController(_repositoryDirectory,
               _workingDirectory);
9         default:
10             throw new NotSupportedException($"Download_{type}_{type.
                  ToString()}_is_not_supported");
11     }
```

12 }

2.4.2 Project Actions

The first usage of this coding strategy can be found in `IProjectAction` interface. This interface is used when initializing a project when the application is opened, where the user can choose between the different actions that can be executed: Create, Open or Import from .zip file. Export action has also been implemented, but it is also accessible through the File Menu, once a project is loaded.

```
1 public interface IProjectAction
2 {
3     public string Title { get; set; }
4     public string ProjectName { get; set; }
5     public string DirectoryPath { get; set; }
6     public bool IsProjectNameVisible { get; set; }
7     public bool IsDirectoryPathVisible { get; set; }
8     public Task<bool> Execute();
9     public Task Browse();
10 }
```

This interface defines five different properties, that are used for initializing the View and ViewModel before showing the new window, and two methods, that correspond to the different actions that can be executed in this window: Browse a directory and Execute.

With this implementation, `IProjectAction` object is used in `HomeProjectActionView-Model`. Only in the constructor the type of action needs to be specified. In the rest of the class, all the Bindings and Commands are executed using this object equally for all the types. A simple example is the implementation of Execute command.

```
1 public ICommand BrowseCmd => new RelayCommand(async _ => await
   BrowseExecute(), CanBrowse);
2
3 private bool CanBrowse(object _)
4 {
5     return true;
6 }
7
8 private async Task BrowseExecute()
9 {
10     _projectAction.DirectoryPath = DirectoryPath;
11     _projectAction.ProjectName = ProjectName;
12
13     await _projectAction.Browse();
14
15     DirectoryPath = _projectAction.DirectoryPath;
16     ProjectName = _projectAction.ProjectName;
17 }
```

In this Command, it can be appreciated the simplicity of the code and the lack of if-else conditions and knowledge of the type of action that is being executed.

2.4.3 Download Controllers

Another class that can take advantage of the use of polymorphism is the DownloadController, which needs a simpler interface.

```
1 public interface IDownloadController
2 {
3     public string RepositoryDirectory { get; set; }
4     public string WorkingDirectory { get; set; }
5     public Task Download(DateTime FromDate, DateTime ToDate, string
6         station);
7 }
```

With this implementation, the only method that needs to be called is Download, but in each implementation, this method execute different actions.

For the moment, only two classes are implemented using this interface, but in the future, if it was desired to add any other data source, it would be possible to implement a new class that implements this interface and used it in the ViewModel as it is done now with the IDownloadController object.

```
1 IDownloadController downloadController = DownloadControllerFactory.
    Create(DownloadType.Earlinet, _earlinetRepositoryDirectory,
        _workingDirectory);
2 await downloadController.Download(FromDate, ToDate, "BRC");
```

2.4.4 Matlab Scripts Implementations

The last interface that has been implemented is IMatlabScript. Running Matlab scripts using C# is not a complicated task, since the use of the library ProcessStartInfo speeds up all the process, needed only the name of the script to be executed. But for this project it was required to execute more actions.

```
1 public interface IMatlabScript
2 {
3     public string Name { get; }
4     public Dictionary<string, object> vars { get; set; }
5     public void PreExecutionActions();
6     public void PostExecutionActions(bool resultOK = true);
7 }
```

It was needed not only to run a script but to have communication between the Matlab scripts and the application. In order to solve this problem, it was decided to create different files:

- Configuration files: Parameters are saved from C# application and read in Matlab before running the corresponding script.
- Output files: Results are saved from Matlab and read in C# application after running the corresponding script, in order to update the UI or execute further actions.

This is very useful, since it allows to have a clear separation between the different tasks and to make the code more maintainable. If it is desired to add a new script, it is only necessary to create a new class using this same interface and call the method for running a Matlab script as in the following example:

```
1 MatlabController.RunMatlabScript(ScriptType.Preview, dict, "_DC");
```

This is possible because this interface is only used for running Matlab Scripts. The MatlabController class is responsible for executing all the sequence for running a script, which consists on:

1. Execute pre-execution actions
2. Run the script (start matlab process)
3. Receive output from Matlab
4. Receive results from Matlab
5. Execute post-execution actions

Each Script implementation will be the responsible for defining the different actions to be executed in each step, taking into account if the script finished with error or was executed successfully.

2.5 Interactions with other applications

The main goal of this application is to create a user-friendly interface for executing all the sequence of actions needed to download data from different web services, combine it and create plots from the results. In order to do that, the application needs to interact with different external applications and web services:

1. Web services: in order to download all the necessary data files from the different sources, Aeronet and Earlinet sites
2. Matlab: in order to execute the different scripts needed for data combination and plotting
3. GRASP algorithm: in order to execute the algorithm and obtain the results

With the use of the different interfaces and classes explained in this section, it is possible to obtain the desired behaviour, which can be summarized in the following diagram:

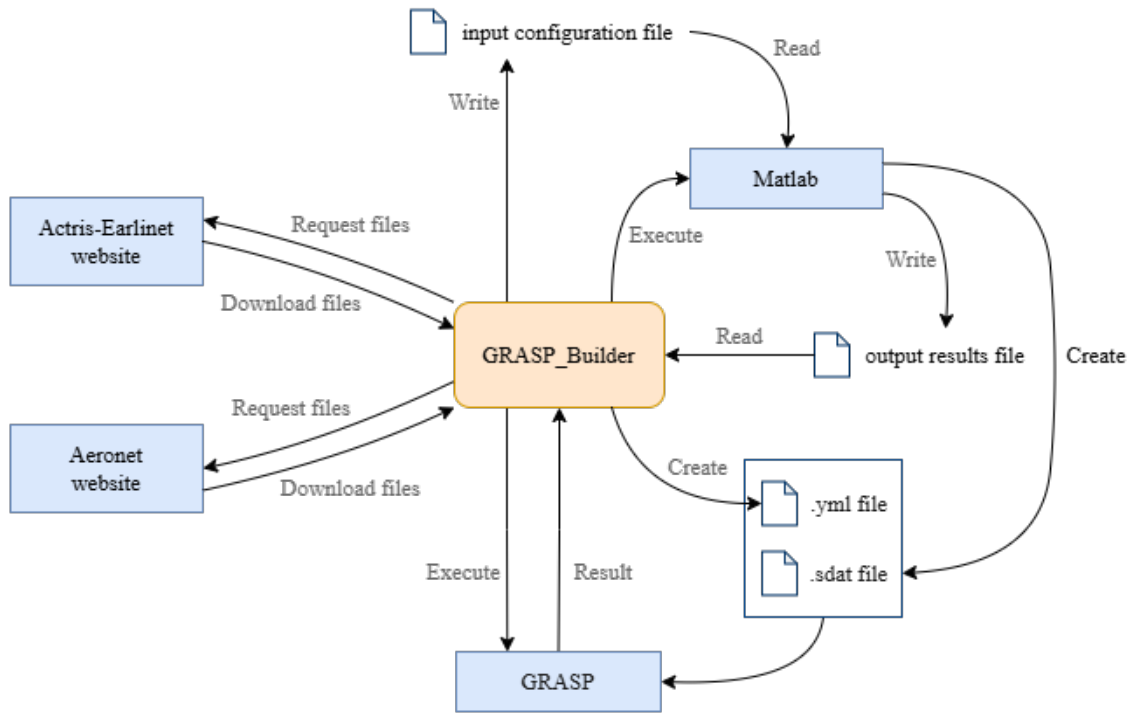


Figure 3: Application interactions diagram

In Figure 3 it can be appreciated all the different actions that are taking place during the execution of the application and the communication between the application and the external applications and web services. All these interactions are not noticed from the user point of view, giving the impression of an easier user experience.

2.6 Code organization

Since this projects has been developed for the CommonSense department, all classes have been organized in different regions in order that the code is easy to understand and maintain for a new developer.

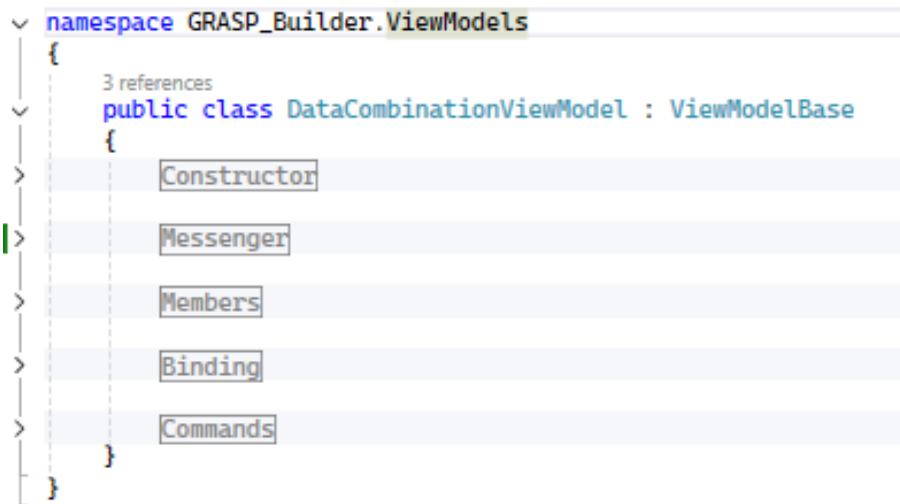


Figure 4: Code organization example

In Figure 4 all the different regions of any View Model class can be appreciated.

- **Constructor:** Only contains the class Constructor method.
- **Messenger:** Contains all the related methods to the different Messenger Register cases that have been defined in the corresponding class.
- **Members:** Contains all the variables and properties of the class. These variables can be accessed by all the class methods.
- **Binding:** Contains all the binding properties implemented for the corresponding View.
- **Commands:** Contains all the commands implemented for the corresponding View.

In addition, different summaries and comments have been added to the code in order to make it easier to understand. The source code can be downloaded from the following repository: <https://github.com/mireialopez0910/TFM/tree/main>. In this repository it is included the GRASP_Builder C# project and both LaTeX projects, the User Manual document and the Master Thesis document, and the User Manual document in PDF format.

Git can be installed through the official website: <https://git-scm.com/download/win>. Once it is installed, the repository can be cloned in the desired folder by executing the following command in the desired terminal:

```
1 git clone https://github.com/mireialopez0910/TFM.git
```


3 Testing

During the development of the application, different tests have been performed to ensure that the application works as expected:

1. First contact:

In order to know how the initial implementation was working and optimise the process it was needed to know how the different steps were being executed and the expected results

- Download files
- Execute Matlab scripts with old data
- Execute GRASP with old data

2. Test downloaded data: After automating the download process, it was needed to test the different matlab scripts with the new data. During these tests, it was detected that some data was being downloaded with a new file format. This was a problem, since the old scripts were not compatible with this new format and they had to be modified.

3. Test GRASP with generated data: After modifying the scripts, it was needed to test the obtained data with the GRASP algorithm. It was detected that the data was not being processed correctly since the configuration file did not match the correct values. This was an issue with file versions that was solved by updating the configuration file to the correct version.

4. Test full UI execution: After defining the application as configurable and workspace independent, it was needed to test the full execution of the application. During this round of tests, it was checked that the project folders were working as expected, improving the project management process.

5. Test full UI execution in CALCULA: Since CALCULA works with LINUX different aspects of the application had to be tested. Some changes were needed for the folders and files management, since some routing was only working in Windows. During these tests the full execution of the application was performed successfully, since it was the only test where the GRASP algorithm could be executed.

6. Test different dates with problematic scenarios: After checking the full execution of the applications, it was needed to test the application with negative tests. Negative test consist on testing the software in known and problematic scenarios, in order to check if the application is able to handle them correctly. The full testing results table can be found in Appendix D. The main cases checked were:

-

Unit testing has also been performed to ensure that the application is working as expected. Unit testing consists in testing the separated components of the application, in this case the Model classes, to ensure that they are working as expected[26].

Implementing this type of testing in a project allows to detect and fix some issues that

were not detected during the integration tests and reduces the time consumption of some testing process.

Test	Duration	Traits	Error Message
UnitTests (13)	2,3 sec		
UnitTests (13)	2,3 sec		
ExecuteMatlab (4)	652 ms		
MatlabController	2 ms		
MatlabScriptFactory_CreatesEx...	650 ms		
SaveFigures	< 1 ms		
SendFiles	< 1 ms		
ManageConfigurations (6)	839 ms		
LoadAppConfig	86 ms		
LoadProjectConfig	15 ms		
ModifyAppConfig	4 ms		
ModifyProjectConfig	20 ms		
SaveAppConfig	18 ms		
SaveProjectConfig	696 ms		
ManageProjects (3)	763 ms		
CreateProject	35 ms		
ImportProjectData	722 ms		
OpenProject	6 ms		

Figure 5: Implemented Unit Tests

In 5 it can be seen the implemented unit tests for this project, where different classes have been tested: application configuration, project configuration and Matlab script management. The main advantages of this type of testing are that they can be performed efficiently, in this case they have been executed within seconds, and can be performed during development. This helps the developer to check that the new code is still working as expected.

4 Conclusions

References

- [1] Wolfgang Skala. Drawing gantt charts in latex with tikz.
- [2] Albert Einstein. Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, 322(10):891–921, 1905.
- [3] Michel Goossens, Frank Mittelbach, and Alexander Samarin. *The L^AT_EX Companion*. Addison-Wesley, Reading, Massachusetts, 1993.
- [4] Donald Knuth. Knuth: Computers and typesetting.
- [5] C.N.R. IMAA. Actris-earlinet data portal. Web Application available at <https://data.earlinet.org/earlinet/desktop.zul>, 2024.
- [6] NASA. Aerosol robotic network (aeronet). Web Application available at <https://data.earlinet.org/earlinet/desktop.zul>, 2025.
- [7] Miriam Martínez Canelo. ¿qué es la programación orientada a objetos? Web Application available at <https://profile.es/blog/que-es-la-programacion-orientada-a-objetos/>, 2025.
- [8] MathWorks. Matlab. Web Application available at <https://es.mathworks.com/products/matlab.html>, 2025.
- [9] C.N.R. IMAA. Actris-earlinet rest api. Web Application available at <https://data.earlinet.org/api/swagger-ui/>, 2025.
- [10] NASA. Aerosol optical depth. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_new.html, 2025.
- [11] NASA. Raw products aerosol optical depth. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_raw_sky_new.html, 2025.
- [12] NASA. Aerosol inversions. Web Application available at https://aeronet.gsfc.nasa.gov/print_web_data_help_v3_inv_new.html, 2025.
- [13] Microsoft. Mvvm. Web Application available at <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>, 2025.
- [14] Microsoft. Observableobject. Web Application available at <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/observableobject>, 2025.
- [15] Microsoft. RelayCommand. Web Application available at <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/relaycommand>, 2025.
- [16] GeeksforGeeks. Solid principles. Web Application available at <https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/>, 2025.

-
- [17] Microsoft. Download visual studio community. Web Application available at <https://visualstudio.microsoft.com/es/vs/community/>, 2025.
 - [18] Microsoft. Download .net. Web Application available at <https://dotnet.microsoft.com/en-us/download/dotnet>, 2025.
 - [19] Microsoft. What is .net? Web Application available at <https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet>, 2025.
 - [20] MathWorks. Download matlab. Web Application available at <https://es.mathworks.com/help/install/ug/install-products-with-internet-connection.html>, 2025.
 - [21] CALCULA. Calcula. Web Application available at <https://ondemand.tsc.upc.edu/pun/sys/dashboard>, 2025.
 - [22] GitHub. Github. Web Application available at <https://github.com/mireialopez0910/TFM/tree/main>, 2025.
 - [23] Microsoft Learn. Polymorphism. Web Application available at <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>, 2025.
 - [24] Microsoft Learn. Factory pattern. Web Application available at <https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/>, 2025.
 - [25] GRASP Open. Grasp. Web Application available at <https://www.grasp-open.com/>, 2025.
 - [26] AWS. Unit testing. Web Application available at <https://aws.amazon.com/es/what-is/unit-testing/>, 2025.

Appendices

A Earlinet Service Class

```
1 public class EarlinetService{
2     string baseUrl = "https://api.actris-ares.eu/api/services/restapi/";
3     public virtual System.Threading.Tasks.Task<bool>
4         DownloadProductByDateRangeAsync(string type, string fromDate,
5         string toDate, string outputFolder){
6         return DownloadProductByDateRangeAsync(type, fromDate, toDate,
7         outputFolder, System.Threading.CancellationToken.None);
8     }
9
10    public async Task<bool> DownloadProductByDateRangeAsync(string type,
11    string fromDate, string toDate, string outputPath, System.
12    Threading.CancellationToken cancellationToken = default){
13        try{
14            string url = $"{baseUrl}products/downloads?kind={type}&
15            fromDate={fromDate}&toDate={toDate}&stations=brc";
16            using (HttpClient client = new HttpClient()){
17                HttpResponseMessage response = await client.GetAsync(url
18                );
19                response.EnsureSuccessStatusCode();
20                using (var fs = new FileStream(outputFilePath, FileMode.
21                Create, FileAccess.Write, FileShare.None)){
22                    await response.Content.CopyToAsync(fs);
23                }
24                System.IO.FileInfo f = new FileInfo(outputFilePath);
25                if (f.Length <= 1048)
26                    return false;
27                return true;
28            }
29        }
30        catch (Exception ex){
31            return false;
32        }
33    }
34 }
```

B Aeronet Service Class

```
1 public class AeronetService{
2     private string baseUrl = "https://aeronet.gsfc.nasa.gov/cgi-bin/";
3     public async Task DownloadDataAsync(string destinationFile, string
4     url){
5         try{
6             using (HttpClient client = new HttpClient()){
7                 var response = await client.GetAsync(url);
8                 response.EnsureSuccessStatusCode();
9             }
10        }
11    }
```

```

9         var content = await response.Content.
10             ReadAsByteArrayAsync();
11         await File.WriteAllBytesAsync(destinationFile, content);
12     }
13     catch (Exception ex){
14         Console.WriteLine($"Error downloading data: {ex.Message}");
15     }
16 }
17
18 public string BuildUrl(DataType _dataType, DateTime startDate,
19     DateTime endDate, string productType = "", string site = "", string
20     product = "", string AVG = "", bool isEnabled = false){
21     switch (_dataType){
22     case DataType.AerosolInversions:
23         if (isEnabled)
24             return BuildUrlAerosolInversions(startDate, endDate,
25                 productType, site, product, AVG);
26         else
27             return BuildUrlAerosolInversions(startDate, endDate);
28
29     case DataType.OpticalDepth:
30         if (isEnabled)
31             return BuildUrlOpticalDepth(startDate, endDate,
32                 productType, site, AVG);
33         else
34             return BuildUrlOpticalDepth(startDate, endDate,
35                 productType);
36
37     case DataType.RawProductsOpticalDepth:
38         if (isEnabled)
39             return BuildUrlRawProductsOpticalDepth(startDate,
40                 endDate, productType, site, AVG);
41         else
42             return BuildUrlRawProductsOpticalDepth(startDate,
43                 endDate, productType);
44     }
45     return "";
46 }
47
48 {...} //BuildUrl methods can be found in GitHub repository
49 }

```

C RunMatlabScript method using ProcessStartInfo class

```

1 public static void RunMatlabScript (string scriptPath){
2     try{
3         ProcessStartInfo startInfo = new ProcessStartInfo{
4             FileName = "matlab",
5             Arguments = $"-batch \"run('{scriptPath}')\"",
6             RedirectStandardOutput = true,

```

```
7         RedirectStandardError = true,
8         UseShellExecute = false,
9         CreateNoWindow = true
10    };
11    using var process = new Process { StartInfo = startInfo };
12    process.OutputDataReceived += (s,e) =>{
13        if (e.Data != null)
14            Messenger.Default.Send<string>("WriteMatlabOutput",e.
15                Data);
16    };
17    process.ErrorDataReceived += (s,e) =>{
18        if (e.Data != null)
19            Messenger.Default.Send<string>("WriteMatlabErrors",e.
20                Data);
21    };
22    process.Start();
23    process.BeginOutputReadLine();
24    process.BeginErrorReadLine();
25    process.WaitForExit();
26    if (process.ExitCode == 0){ //OK
27        Logger.Log($"Script was executed correctly, executing post_
28            execution_actions...");
29        script.PostExecutionActions();
30    }
31    else{
32        Logger.Log($"Error occurred during execution, executing post_
33            execution_actions...");
34        script.PostExecutionActions(false);
35    }
36 }
37 }
```

D Test Results